

Multi-objective Aircraft Landing

In this assignment, we want to **both** minimize the arrival target penalty **and** maximize the worst case separation time between all airflights. To achieve this we maximize the negative target penalty and maximize the worst-case separation, the variable W .

Problem

- Maximize the negative target time penalty and maximize the worst case (smallest) separation time.

New decision variable

- The smallest landing time difference between two adjacent planes : $W \geq 0$.

Model

Maximize.

$$-\sum_{i \in I} \delta_i e_i + \rho_i l_i$$

W

Subject to :

$$t_i + Sep_{i,j} + W - (M + Sep_{i,j}) \cdot (1 - x_{i,j}) \leq t_j \quad \forall i, j$$

... and the other constraints

```
*****
# MO Aircraft Landing, 1 assignment, "Mathematical Programming Modelling" (42112)
using JuMP
using Gurobi
using HiGHS
using MultiObjectiveAlgorithms
using Plots
*****
```

```
*****
```

```

# PARAMETERS
EA    = [125 187  94  99 111 123 129 131 142 155]
TTime = [155 212 134 127 123 138 145 157 162 179]
LA    = [473 612 508 421 511 515 525 407 500 612]

EP=[10 20 35 25 35 30 30 50 30 35]
LP=[10 20 35 25 35 30 30 50 30 35]

Sep = [0 5 15 15 15 15 15 10 15  5;
       5 0 15 15 10 15 10 15 15 15;
       15 15 0 10 15 10 15 10 15 10;
       15 15 10 0 10 10 10 15 15 15;
       15 10 15 10 0 10 5 10 15 5;
       15 15 10 10 10 0 15 15 15 15;
       15 10 15 10 5 15 0 10 5 10;
       10 15 10 15 10 15 10 0 10 15
       15 15 15 15 15 15 5 10 0 10
       5 15 10 15 5 15 10 15 10 0];

n=10
M=maximum(LA)-minimum(EA)
*****

*****
# Model
model = Model()

#binary sequence variables
@variable(model, x[i=1:n,j=1:n], Bin)
#arrival time variables
@variable(model, t[i=1:n]>=0, Int)
#the number of minutes plane i arrives early
@variable(model, e[i=1:n]>=0, Int)
#the number of minutes plane i arrives late
@variable(model, l[i=1:n]>=0, Int)
#the position of aircraft i in the landing sequence
@variable(model, p[i=1:n]>=0)
#separation variable
@variable(model, W>=0)

@expression(model, sep, W)
@expression(model, pen, -sum(EP[i]*e[i]+LP[i]*l[i] for i=1:n))
@objective(model, Max, [sep, pen])

@constraint(model, [i=1:n], t[i]==TTime[i]-e[i]+l[i])

```

```

@constraint(model, [i=1:n], l[i]<=LA[i]-TTime[i])
@constraint(model, [i=1:n], e[i]<=TTime[i]-EA[i])

@constraint(model, [i=1:n,j=1:n; i!=j], t[i]+Sep[i,j]+W<=t[j]+(Sep[i,j]+M)*(1-x[i,j]))
@constraint(model, [i=1:n,j=1:n; i!=j], x[i,j]+x[j,i]==1)

@constraint(model, p[1]==sum((1-x[1,j]) for j=2:n)+1)
@constraint(model, [i=2:n-1],
    p[i]==(sum(x[k,i] for k=1:i-1)) +
    sum((1-x[i,k]) for k=i+1:n)+1)
@constraint(model, p[n]==sum(x[j,n] for j=1:9)+1)

#5/6 not consecutive
@constraint(model, p[6] >= p[5]+2-(n+1)*(1-x[5,6]))
@constraint(model, p[5] >= p[6]+2-(n+1)*x[5,6])
#####

#####
# solve

set_optimizer(model,
    () -> MultiObjectiveAlgorithms.Optimizer(Gurobi.Optimizer))
set_attribute(model,
    MultiObjectiveAlgorithms.Algorithm(),
    MultiObjectiveAlgorithms.EpsilonConstraint())

optimize!(model)
#####

#####
# Report results
if result_count(model)>=1
    println("No Pareto optimal points: ",result_count(model))
    for r in 1:result_count(model)
        z = objective_value(model; result = r)
        pp=round.(Int,value.([p[i] for i=1:n]; result = r))
        println("Sep: ",round.(Int,value.(z[1])), "    Penalty: ",
            -1*round.(Int,value.(z[2])), "    aircraft pos =", pp)
    end
else
    println("No solutions found")
end
#####

```